PRODUCT INTERN ASSIGNMENT

KURATE — Product, Systems, Quality & Analytics Challenge

Submitted by:
Afreen Laiq

B.Tech — Computer Science & Engineering

Role: Product Intern

About Me

I am a Computer Science & Engineering graduate with a strong interest in product thinking, user experience, problem-solving, and technology. Through this assignment, I have explored how a product idea can be framed, built, tested, and evaluated using data.

My approach throughout this case study has been to focus on user needs, practical solutions, quality, and evidence-based decision-making.

Acknowledgement

I would like to thank the **WTF team** for providing me with the opportunity to work on this Product Intern challenge. The assignment provided a valuable opportunity to understand product decisions beyond just building features, and to think across product, technology, QA, and analytics.

I appreciate the opportunity to present my thinking through this case study.

Thank You

Mission 01: Trust Discovery

Trusted Search

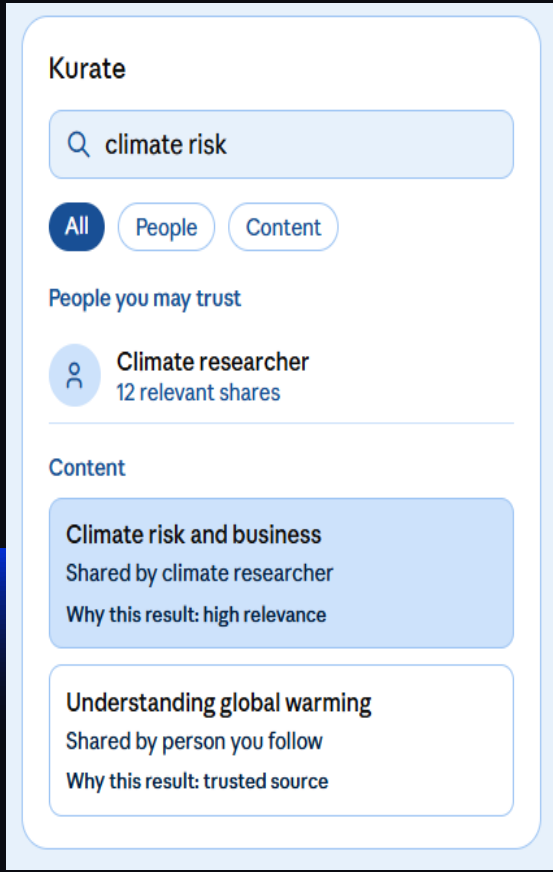
Helping Kurate members discover useful content and trusted people not just popular results.

User & Problem	Principles	Features
Primary user: Curious, information-seeking Kurate members who regularly consume articles, videos, and newsletters and want to discover trustworthy information through people they trust.	**TRUST FIRST** • Relationship and provenance matter more than popularity.	**Trusted Search(P0 Priority)** • Help users quickly find trustworthy knowledge.
Job-to-be-done: When I want to understand a topic, I want to quickly find relevant and trustworthy content and people through my trusted network, so I can learn without filtering through a large amount of generic or low-value information.	**USEFUL > ENGAGEMENT** • Optimize for useful discovery, not time spent.	**People-based Discovery(P1 Priority)** • Help users discover *who to learn from*, not only *what to read*.
Problem: Kurate users may have difficulty finding the right information or knowledgeable people when they want to explore a topic because content discovery can become noisy, while conventional ranking can favor engagement rather than trust and usefulness.	**EXPLAINABLE** • Users should understand why a result appears. **FRESH + DIVERSE** • Keep results current without creating a repetitive filter bubble.	**Trusted Context(P2 Priority)** • Help users judge whether a result is worth opening.

Chosen Feature: Trusted Search

Reason: It directly addresses the core problem of finding useful content and trusted people while providing a clear foundation for the remaining product, engineering, QA and analytics work.

1.1 Wireframe – Trusted Search



1.2 Ranking Logic

Ranking Signals:

- Query/content relevance
- Trust relationship with the source/person
- Content quality
- Recency
- User engagement as a secondary signal

Cold Start:

For new users with limited history, rank primarily using relevance, quality and broad trust signals until sufficient user behaviour is available.

Personalization Risk:

Over-personalization may create a filter bubble, repeatedly showing similar viewpoints and reducing discovery of diverse perspectives.

1.3 Metrics

- **Primary Success Metrics:**

Useful Search Rate
= Searches resulting in a result opened within 60 seconds ÷ Total searches
Measured over 7 days.

- **Guardrail Metrics**

Zero-Result Rate
= Zero-result searches ÷ total searches
Measured over 7 days.

Search Abandonment Rate
= Searches with no result opened within 60 seconds ÷ total searches
Measured over 7 days.

Mission 02: Buildable System

2.1 Scope & Non-Goals

In Scope:

- Search articles, videos, newsletters and relevant people.
- Rank results using relevance, trust, quality and recency.
- Show why a result is relevant.
- Track search and result-opening events.

Non-Goals:

- Replace Google as a general-purpose search engine.
- Build a complete social recommendation system.
- Optimize ranking only for clicks or time spent.

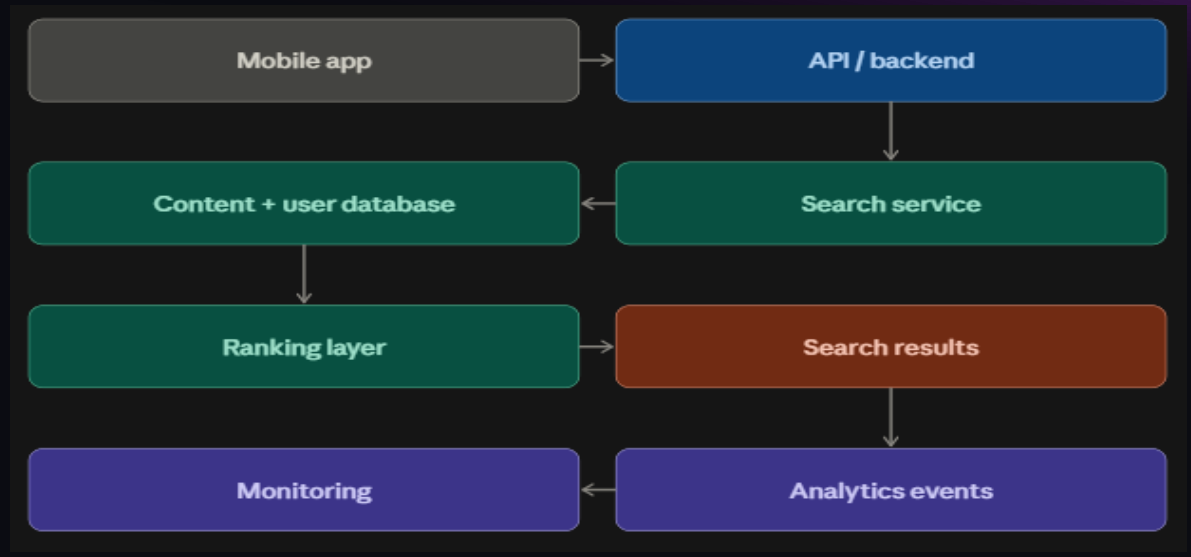
2.2 User Flow

2.3 Technical Framework

Layer	Choice	Reason
Mobile Client	React Native	Reusable mobile UI and existing JS ecosystem
Backend	Node.js + Express	Lightweight API layer
Database	PostgreSQL	Structured user/content relationships
Search	Elasticsearch/OpenSearch	Supports keyword, relevance and ranking
Analytics	Event-based tracking	Measures search behaviour and outcomes

Tradeoff: A dedicated search engine adds operational complexity but provides stronger search and ranking capabilities than database-only search.

2.4 System Architecture



2.5 Search Approach

Approach	Advantage	Limitation
Keyword	Fast, predictable exact matching	Misses related concepts/synonyms
Semantic	Understands meaning beyond exact words	More complex; may return unexpected results
Hybrid — Chosen	Combines exact matching with semantic relevance	Higher implementation complexity

Chosen approach: Hybrid Search

It combines keyword relevance with semantic similarity, then applies Kurate-specific signals such as **trust, quality and recency**.

2.6 Failures States

State	User Experience
Loading	“Searching Kurate…”
Empty	“No relevant results found. Try a different search.”
Partial	Show available results and indicate that some results may be unavailable.
Offline	“You’re offline. Please reconnect and try again.”
Permission Denied	Explain why access is required and provide the appropriate action.
Error	“Something went wrong. Please try again.”

2.7 Data, Privacy & Abuse Control

Data to store

- User ID
- Search query
- Search timestamp
- Result IDs shown
- Result opened
- Search outcome

Data to avoid unnecessarily storing

- Sensitive personal information unrelated to search functionality.

Privacy / Abuse Control

- Apply **rate limits** to search requests to reduce automated abuse and excessive traffic.

Mission 03: Release Quality

3.1 Bug / Issue Reports

Bug A — Duplicate Share

Context: Android 14; Chrome link shared to Kurate; app backgrounded and reopened during network switching.

Steps:

- Open a link in Chrome.
- Share it to Kurate.
- Switch network/background the app.
- Return to Kurate.
- Observe the composer.

Expected: One composer and one share action.

Actual: Composer opens twice; duplicate posts occurred in 3/5 attempts.

Reproducibility: 3/5 attempts.

Impact: Duplicate content and possible data/analytics corruption.

Severity: High.

Evidence Needed: Device/build details, network logs, request IDs, backend logs.

Regression Area: Share flow, deep linking, network recovery, duplicate-request handling.

Bug B — Search Returns No Results

Context: Searching **"climate risk"** sometimes returns no results even though relevant content uses **"global warming."**

Steps:

- Search for "climate risk."
- Observe results.
- Repeat under different network/cache conditions.

Expected: Relevant content using related terminology should appear.

Actual: Zero results appeared in 4/10 observations; results sometimes appeared after ~10 seconds.

Reproducibility: 4/10 observations.

Impact: Users may fail to discover relevant content.

Severity: Medium–High.

Evidence Needed: Search logs, indexing timestamps, cache state, query/result logs.

Regression Area: Search indexing, synonyms, ranking, caching.

Bug C — Accessibility

Context: Pixel 7, text size set to 200%.

Steps:

- Open onboarding.
- Increase system text size to 200%.
- Navigate to the Continue button.

Expected: Continue remains visible and usable.

Actual: Button moves below the visible screen.

Reproducibility: 5/5.

Impact: User may be unable to complete onboarding.

Severity: High.

Evidence Needed: Device/OS version, screenshots, accessibility settings.

Regression Area: Onboarding layout, responsive UI, accessibility.

3.2 Test Matrix

Area	Test Case	Expected Result
1. Happy Path	Search a valid query	Relevant results displayed
2. Search	Search with no matching content	Correct empty state
3. Search	Search related terms/synonyms	Relevant results returned
4. Network	Network switches during search	Search recovers without duplicate or incorrect results
5. Network	Search while offline	Clear offline state
6. Share	Share the same link once	Only one post created
7. Recovery	Reopen app after interruption	User returns to correct state
8. Accessibility	Use 200% text size	All controls remain visible/usable
9. Permissions	Deny required permission	Clear explanation/recovery option
10. Data Quality	Result contains valid title/source	Correct information is displayed
11. Abuse	Send repeated search requests	Rate limiting prevents abuse
12. Error Recovery	Search service fails	Graceful error + retry option

3.3 Release Gates

Local → Pull Request → CI → Staging → Beta → Production

- **Local:** Unit tests + linting
- **Pull Request:** Code review + targeted tests
- **CI:** Automated test suite + successful build
- **Staging:** Integration + regression testing
- **Beta:** Limited-user validation + monitoring
- **Production:** Gradual release + monitoring

3.4 Stop-Ship & Rollback

Stop-ship if:

- Duplicate posts can be reliably created from one user action.
- Users are blocked from completing onboarding.
- Critical data loss or privacy/security issues are detected.

Rollback trigger:

- A critical production issue appears or the primary search metric deteriorates significantly without an acceptable explanation.

3.5 Production Signals

- Useful Search Rate
- Zero-Result Rate
- Search Error Rate
- Duplicate-Share Rate
- Crash/Error Rate

Mission 04: Product Learning

4.1 Priority Product Problems

Problem 1 — Search discovery is weak

- **287** users were eligible.
- 142 saw search, but only 67 searched.
- Only 31 opened a result within 60 seconds.
- 18 sessions had zero results.

Evidence limit: We cannot conclude that search relevance is poor from zero-result sessions alone because indexing lag, synonym behavior, cache state and timestamps are unknown

Problem 2 — Users are not consistently reaching value

- 451 users completed onboarding.
- 287 followed at least 3 people.
- 196 opened at least 2 items.
- Only 74 performed a defined “meaningful action.”
- Only 42 returned on Day 7.

Evidence limit: The “meaningful action” definition may not fully represent user value, and the data does not establish why users fail to return.

4.2 Prioritized Hypotheses

Priority	Hypothesis	Impact	Confidence	Effort
H1	Improving trusted search relevance will increase successful result discovery.	High	Medium	Medium
H2	Improving the onboarding path to trusted people will increase early product value.	High	Medium	Medium
H3	Improving the Reader experience will increase downstream engagement.	Medium	Low	High

Chosen For testing: H1 — Improve Trusted Search relevance.

Priority: Search is prioritized because it is directly tied to the chosen feature and has a measurable discovery funnel with clear optimization opportunities.

4.3 Experiment — Improve Trusted Search

Hypothesis:
Improving search relevance will increase the number of users who successfully discover useful content.

Eligible Users:
Users who have access to Trusted Search and perform at least one search during the experiment.

Assignment:
Randomly split eligible users into:

- **Control:** Current search experience
- **Treatment:** Improved hybrid search + trust/relevance ranking

Confirmed Exposure:
User receives search results from the assigned experience.

Primary Outcome:
Useful Search Rate = Searches resulting in a result opened within 60 seconds ÷ Total searches

Guardrails:
- Zero-result rate
- Search error rate
- Search latency

Time Window: 7 days.

Decision Rule: Ship the treatment if Useful Search Rate improves meaningfully without a significant deterioration in guardrail metrics.

4.4 Event Dictionary

Event	Trigger	Required Properties	Validation
Search_started	User submits query	user_id, query_id, query	Compare event count with search requests
search_results_shown	Results displayed	query_id, result_count, variant	Verify results rendered
result_opened	User opens result	query_id, result_id, timestamp	Match with UI action
zero_results	No results returned	query_id, variant	Compare with search response
search_error	Search fails	query_id, error_type, variant	Match backend error logs

Additional Research Before Changing Low-Use Features

Before removing or significantly changing a low-use feature, conduct:

- **5–8 user interviews** with users who tried and did not use the feature.
- **Usability testing** to identify where users struggle.
- **Segmentation** by new vs returning users and highly engaged vs low-engaged users.
- Check whether low use is caused by **poor discoverability, low relevance, technical failure, or genuinely low user need**.

Evidence Limit: Low usage alone is not sufficient reason to remove a feature; we need to separate **eligibility → exposure → attempted use → successful use → downstream value**.

AI-Use Disclosure

AI tool used: ChatGPT & Claude AI.

Use: ChatGPT was used to assist with structuring, wording, test-case generation, product reasoning, and analysis of the provided assignment data. Claude was used to assist with creating the diagrams included in the submission.

Verification: I used AI as a supporting tool to make the work more efficient. The analysis, reasoning, prioritization, and final decisions were done by me. I reviewed the AI-assisted output against the assignment requirements and checked the key calculations and assumptions before finalizing the submission.